



Software Management Tools

Amitabha Sanyal

April 2026

This file is meant for personal use by cgodhandaraman@gmail.com only.
Proprietary content. ©Copyright protected. All Rights Reserved. Unauthorized use or distribution prohibited.
Sharing or publishing the contents in part or full is liable for legal action.



GIT

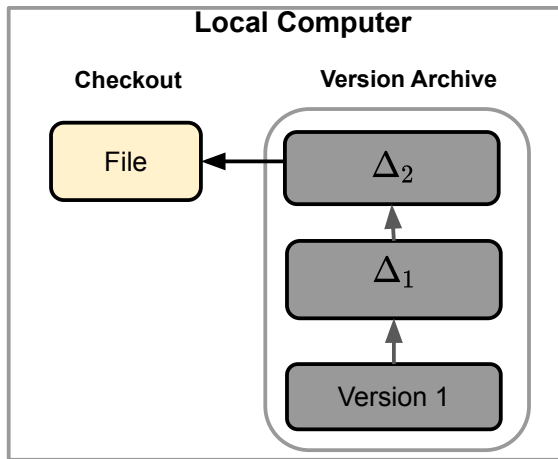


What is a version control system?

- Software is built
 - Incrementally
 - In collaboration
 - As more than one independent strands of development.
- Version control is a system that records changes to a set of files over time so that you can recall specific versions later.

- **Checkout** - the version that you are currently working on
- **Patch set** - The difference (delta) between one version and another

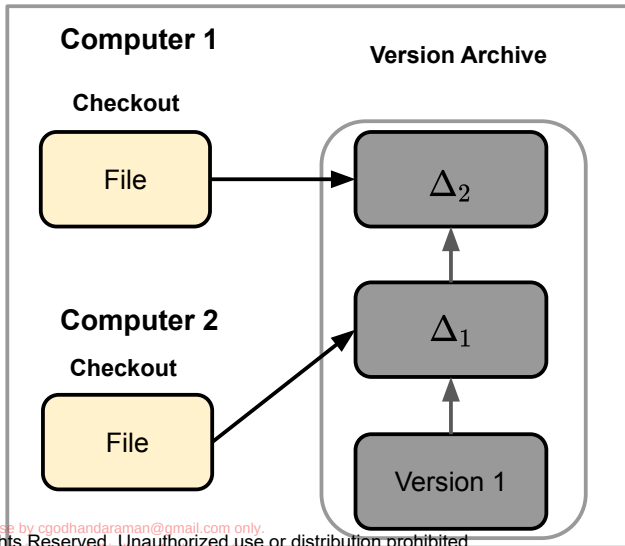
A local version control system (rcs)





What is a version control system?

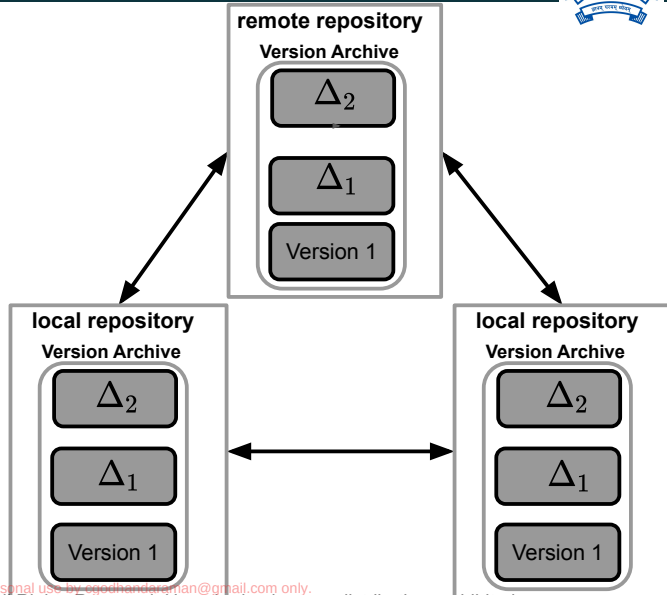
- A centralized version control system (cvs, svn)
 - Possibility of collaboration.
 - Centralized server is vulnerable.





Distributed version control

- Each client fully mirrors the repository.
- Vulnerability of centralized server is minimized.
- If the server dies, any of the clients can be copied back.



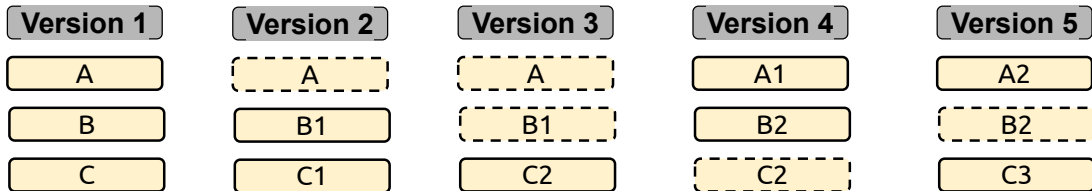


- **Design goals:**
 - Simple design
 - Speed
 - Strong support for non-linear development (thousands of parallel branches)
 - Fully distributed
- Able to handle large projects like the Linux kernel efficiently.

git characteristics



commits over time →



- Every commit is a snapshot of a full record of all the files.
- Most operations are local.
- Git has integrity. Everything in git is SHA-1 hashed. A file or a directory is referred by the hash value.

Example hash value: **4b9da6552252987aa493b52f8696cd6d3b000373**

This file is meant for personal use by cgodhandaraman@gmail.com only.

Proprietary content. ©Copyright protected. All Rights Reserved. Unauthorized use or distribution prohibited.



Setting up git

- > `sudo apt-get install git`
- > `git config --global user.name "<user name>"`
- > `git config --global user.email <email address>`
- > `git config --global core.editor emacs`
- > `git config --global merge.tool meld`
- > `git config --list`

`user.email=<email address>`

`user.name=<user name>`

`core.editor=emacs`

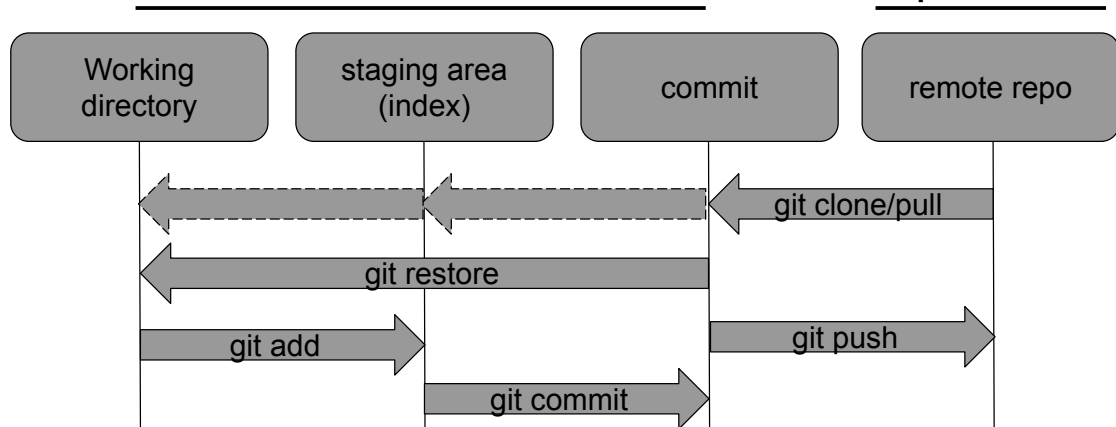
`merge.tool=meld`

`...`

- > `mkdir git-experiments; cd git-experiments`
- > `git init`

local operations

non-local operations



git - basic commands



```
> touch file1.txt      # create file1.txt
> touch file2.txt      # create file2.txt
> git add file1.txt    # stage  file1.txt
> git status           # show the state of git
>
```

On branch **master**

No commits yet

Changes to be committed:

(use "git rm --cached <file>..." to unstage)

new file: **file1.txt**

Untracked files:

(use "git add <file>..." to include in what will be committed) **file2.txt**

git - basic commands



- > `git commit file1.txt -m "Committing the file file1.txt"`
[master (root-commit) 91bb7e6] Committing the file file1.txt
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 file1.txt
- > `git status` # show the state of git
On branch master
Untracked files:
 (use "git add <file>..." to include in what will be committed)
 file2.txt
- > `git log`
commit 91bb7e6712ba65786976a7d23d88b0a8269b6044 (HEAD -> master)
Author: Amitabha Sanyal <amit23358@gmail.com>
Date: Tue Aug 18 21:16:18 2020 +0530

Committing the file file1.txt



Three areas of git

- create `file.txt`

Working area	Staging area	Commit
<code>file.txt - v1</code>		

- `git add file.txt`

<code>file.txt - v1</code>	<code>file.txt - v1</code>	
----------------------------	----------------------------	--

- `git commit -m "msg"`

<code>file.txt - v1</code>	<code>file.txt - v1</code>	<code>file.txt - v1</code>
----------------------------	----------------------------	----------------------------

- modify `file.txt`

<code>file.txt - v2</code>	<code>file.txt - v1</code>	<code>file.txt - v1</code>
----------------------------	----------------------------	----------------------------



Three areas of git

- `git add file.txt`

Working area	Staging area	Commit
<code>file.txt - v2</code>	<code>file.txt - v2</code>	<code>file.txt - v1</code>

- `modify file.txt`

<code>file.txt - v3</code>	<code>file.txt - v2</code>	<code>file.txt - v1</code>
----------------------------	----------------------------	----------------------------

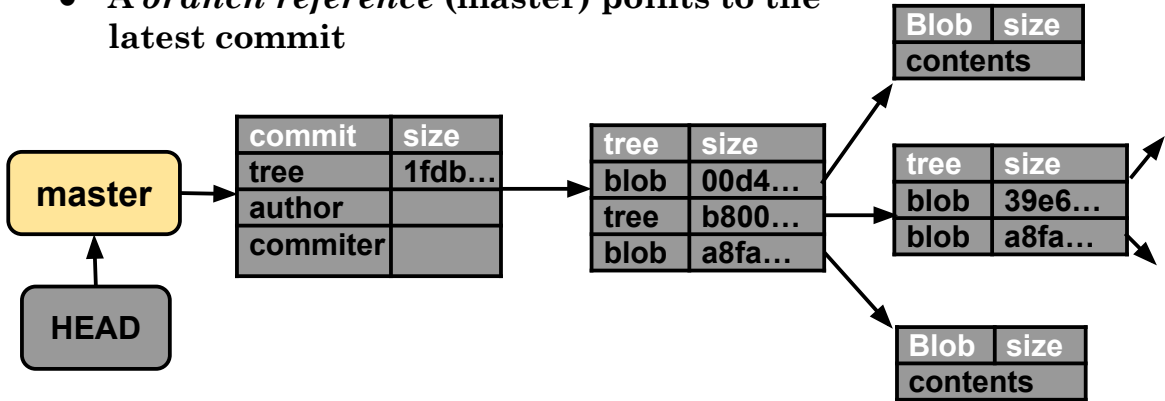
- `git commit -m "msg1"`
`git commit file.txt`
`-m "msg1"`

<code>file.txt - v3</code>	<code>file.txt - v2</code>	<code>file.txt - v2</code>
<code>file.txt - v3</code>	<code>file.txt - v3</code>	<code>file.txt - v3</code>



Git objects and their structures

- Git objects consist of commits, trees and blobs
- Each object is named by its hash value.
- A *branch reference* (master) points to the latest commit





- The output of
`$ diff file1 file2`

Can be interpreted as an answer to:

"What changes do we have to make to file1 to make it identical to file2"

- To interpret the result of a diff, we have to identify a notion of a *before file* (file1, in this case) which, through a set of changes, is made identical to an *after file* (file2).
- What is the corresponding notion in `git diff`?



git diff --no-index

file1.txt

A line has been added
A second line has been added.
A third line
A fourth line
The last line

file2.txt

To illustrate diff adding a
new line
A line has been added
A second line has been added.
And another line here
A fourth line

```
> git diff --no-index file1.txt file2.txt
```

...

```
@@ -1,5 +1,5 @@
```

```
+To illustrate diff adding a new line -- Added line
```

```
A line has been added -- line not changed
```

```
A second line has been added. -- line not changed
```

```
- A third line -- line deleted
```

```
+ And another line here -- Added line
```

```
A fourth line -- line not changed
```

```
-The last line
```

Variations in git diff



command	compares	before	after
<code>diff --no-index file1 file2</code>	file1 vs file2	file1	file2
<code>git diff</code>	Index vs Working Dir	Index	Working Dir
<code>git diff --cached</code>	HEAD vs Index	HEAD	Index
<code>git diff HEAD</code>	HEAD vs Working Dir	HEAD	Working Dir
<code>git diff commitA commitB</code>	CommitA vs CommitB	CommitA	CommitB



The log of all commits-- `git log`

Common `git log` options

- `git log --oneline`
shows each commit in one line (useful for quick overview)
- `git log --graph`
shows a visual branch structure
- `git log -p`
shows diffs introduced by each commit
- `git log file.txt`
shows history of a specific file

useful alias:

```
alias gitlog='git log --oneline --graph'
```

Undoing...



- **Undoing commit**

- > `git commit --amend`

Overwrites the current commit with subsequently staged files and a new message.

- **Undoing staging**

- > `git reset HEAD <file>`

Brings back <file> to the modified state (commit → index)

- **Undoing modification**

- > `git checkout <file>`

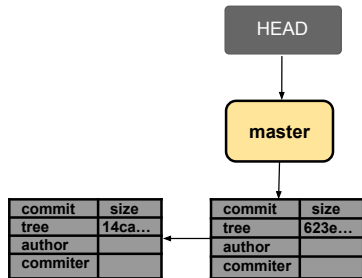
Replaces the modified file with the last committed version of <file> (commit → working dir).

- **These are dangerous operations done under the guidance of trained professionals. Do not try these at home!!!**



Branching

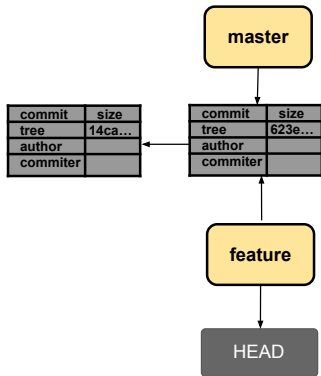
- A single branch is a sequential line of development
- A new branch starts a new line of development that does not interfere with the original.
 - Enables parallel development of new ideas
 - Eventually, may be merged with the original branch
- Commits in a branch are backward chained
- A *symbolic reference* (HEAD) points to the current branch.



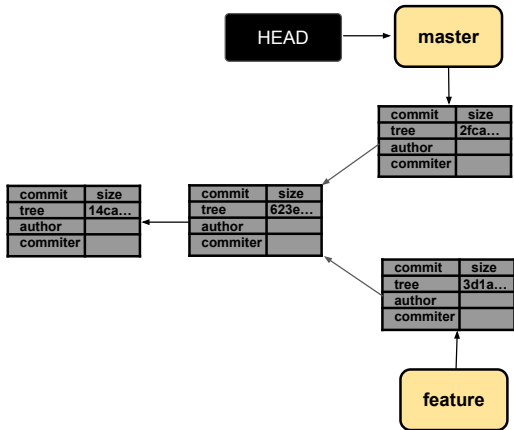


Separate developments

Introduction of a new branch named feature



Separate development along branches



- A change of branch also changes the files visible in the working directory

Useful branch commands



- **git checkout -b bname**
Introduces a new branch **bname** which becomes the current branch
- **git checkout bname**
makes an existing branch **bname** the current branch
- **git branch**
lists all branches
- **git branch -d bname**
deletes the branch **bname**

git stash



If we want to change branch without committing, to preserve the current status, we have to **stash**.

> **git stash**

stashes or stores the current working area and staging area on a stack

> **git stash list**

show the stack of stashes

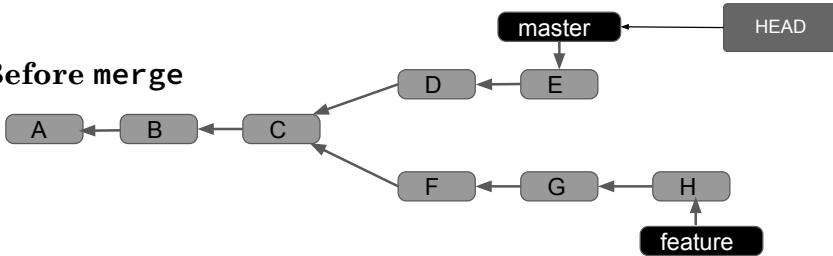
> **git stash apply --index stash@{n} --index**

restores **n**th stash from the stack

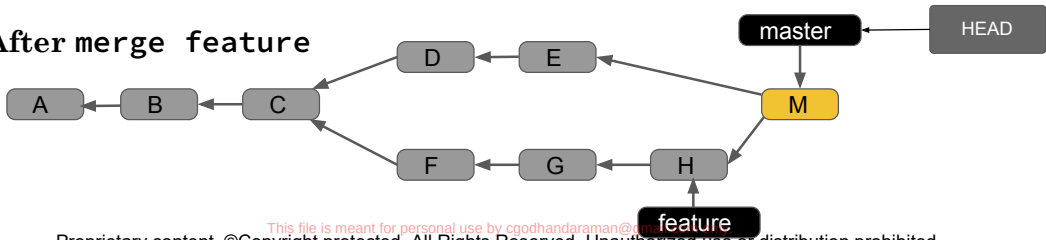
git merge



- Before merge



- After merge feature



git merge



- **Merges the histories of two branches**
- **Creates a merge commit with two parents, preserving both lineages**
- **Does not rewrite history**
- **Typical workflow:**
 - `$ git checkout master`
 - `$ git merge feature`
- **Result**
 - **master advances to a new commit combining master and feature**
 - **The original feature commits remain intact**
- **Notes:**
 - **If there are conflicts, Git will pause and ask you to resolve them**
 - **Use `git log --graph --oneline --all` to visualize merges**

git merge



- `git merge <branch>`

Merges the given branch into the current branch.

- `git merge --continue`

Continues a merge after resolving conflicts.

- `git merge --abort`

Cancels a conflicted merge and resets the state to before the merge.

- `git log --merge`

Shows commits that are causing the merge conflict.

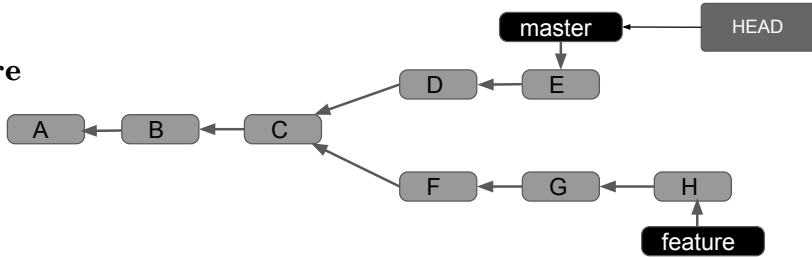
- `git diff <branch>`

Shows what would be merged from that branch.

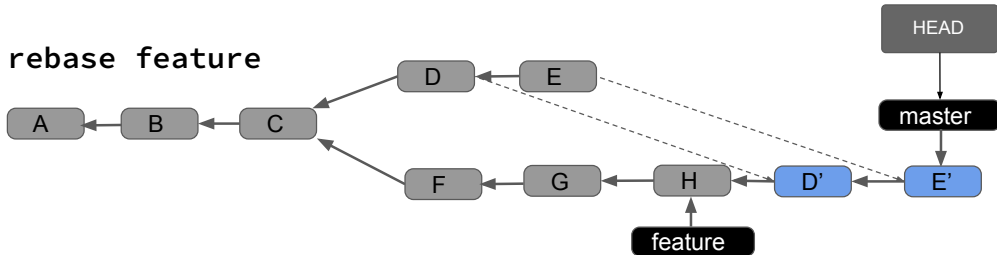
git rebase



- Before



- After rebase feature



git rebase



- Moves or replays commits from one branch onto another
- Produces a linear history — no merge commits
- Rewrites history by creating new commit hashes
- Typical workflow:
 - \$ git checkout master
 - \$ git rebase feature
- Result
 - Commits from master are rebased on top of feature
 - Makes the project history cleaner and easier to follow
- Notes:
 - If there are conflicts, Git will pause and ask you to resolve them
 - Use git log --graph --oneline --all to visualize merges



Working with remote repositories

 Bitbucket

Your workPull requestsRepositoriesProjectsPeopleMore ▾

Create ▾

Q Search

epgd-2025

 Repositories

 Project settings

amit23358 / Projects / epgd-2025

Repositories

Name	Size	Last updated
 git-experiments	9.9 MB	42 minutes ago
 PDT	237.7 MB	42 minutes ago



Working with remote repositories

- `git remote add origin git@bitbucket.org:amitabhasanyal/git-experiments.git`
 - Links the local repo with the remote repo. Also gives it a name “origin”
- `git fetch origin`
 - Brings in meta-data but not files
- `git push -u origin master`
 - Pushes the local repo to the remote repo at branch origin
- `git pull -u origin master`
 - Pulls in files from remote repo to local repo at branch origin

When we want to start with an existing remote repo

- `git clone git@bitbucket.org:amitabhasanyal/git-experiments.git`
 - Clones an existing project called `git-experiments.git` into local repo
 - Checks out the default branch (usually main or master)
 - Does `git remote add origin` to the `git-experiments` URL.

Conclusion



- **Git is essentially a tool for tracking the evolution of code in a project.**
- **The evolution is modeled as a directed acyclic graph (DAG) of commits.**
- **A commit in git is a complete snapshot of the project's state and not merely the difference with the earlier snapshot.**
- **Branches allow concurrent strands of development.**
- **Merges and rebases are different ways of merging. Merge preserve history, rebase rewrites it for a linear history.**
- **The git way: Work locally and independently. Sync when ready by pushing.**

Resources



Books

- Pro Git by Scott Chacon and Ben Straub: <https://git-scm.com/book>. Some of the material in these slides are based on this book.
- Git Pocket Guide: A Working Introduction, by Richard E. Silverman, O'Reilly Media, 2013.

Official Documentation

- <https://git-scm.com/docs>: Authoritative and up-to-date.

Interactive Tutorials

- [Learning git branching](#): A nice hands-on tutorial.
- [GitHub Skills](#) — Git and GitHub workflows.
- [Bitbucket Git Tutorials](#) — Yet another tutorial.



Docker

Amitabha Sanyal

June 1, 2026

What is Docker and why is it useful?



- Assume that you have developed an app on Python 3.5.2 under Ubuntu 16.04.
- On deployment, some users complain that the app fails to run.
- Investigation show that their machines run a higher version of Python, and this is the problem code:

```
import time

try:
    print(time.clock())
except AttributeError:
    print("time.clock() is not available in this Python version")
```

Output with Python 3.5.2: **0.036259**

Output with Python 3.10.12: **time.clock() is not available in this Python version**

- Environment changes frequently lead to behavioral differences in software.

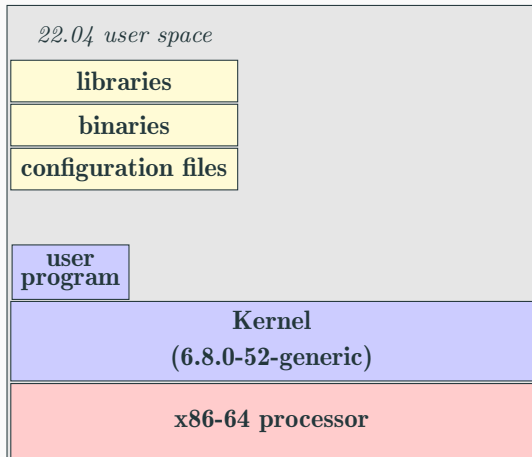


Problem Statement:

- An app has been developed on Ubuntu 16.04 with its own **user space**.
- It must run seamlessly on various Unix-like systems (e.g., Ubuntu 22.04).
- Question: How does Ubuntu 22.04 provide the capabilities to run the app?
- Question: How can it ensure isolation between the app and the host OS?

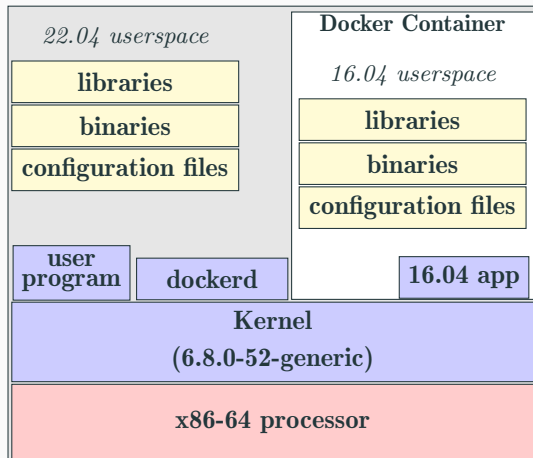
High Level Solution:

- The host system provides a sandboxed environment called a container.
- The container has a minimal set of 16.04 compatible libraries and binaries which are expected to run under the host kernel.
- Isolation is ensured by separating process, filesystem, and network namespaces.



The Ubuntu 22.04 OS consists of:

- The kernel that provides abstractions in the form of system calls for memory management, IO and hardware components.
- The Ubuntu 22.04 userspace:
 - libraries, e.g. in `/lib/x86_64-linux-gnu/` and `/usr/lib/x86_64-linux-gnu/`
 - binaries, e.g. in `/bin/`, `/usr/bin/`, `/usr/sbin/`, etc.
 - configuration files (usually under `/etc`) that applications depend on.



The Dockerized Ubuntu 16.04 consists of:

- A minimal userspace:
 - Essential libraries: **glibc**, **libstdc++**, **libm**, libraries for threading and linking, networking and security.
 - Core shell and command utilities, text processing, networking, system management and process control.
 - Essential configuration files.
- The container ensures isolation from the host machine. The containerized app sees only the minimal userspace.



Filesystem Isolation

- The app sees only Ubuntu 16.04 files, not the host's files.
- Filesystem operations inside container do not touch the host system.

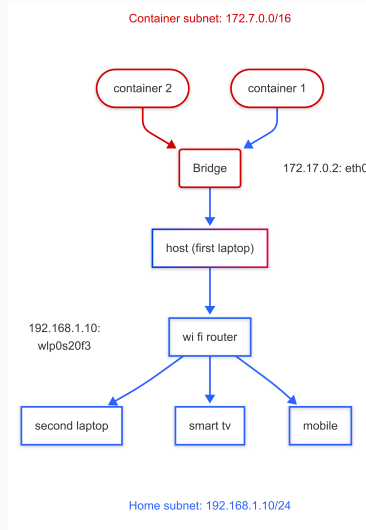
Process Isolation

- PID numbers inside the container come from a different PID namespace than on the host. PID 1 inside the container is different from PID 1 on the host.
- A process inside the container cannot send signals (such as **kill**) to processes outside its own PID namespace. But the converse is possible: the host can signal container processes!

Network Isolation

- The container has its own network interfaces and IP addresses. It cannot access the network interfaces of the host.
- Devices on the host network cannot send messages to a container. However, the container can expose ports that map to the host, allowing external access.

The container subnet





- **System Call Compatibility:** The host kernel must support system calls by the application, e.g.
 - Memory management: `mmap`, `brk`
 - Process control: `fork`, `clone3`
 - Filesystem operations: `open`, `read`, `write`, `close`
 - Networking operations: `socket`, `connect`, `send`, `recv`
 - Signal delivery: `kill`, `sigaction`
- **ABI compatibility:** The kernel ABI must be compatible with the ABI convention of the app:
 - Function calling conventions,
 - Data types and sizes and alignments,
 - Endianness.
 - Executable formats (e.g., `ELF`).

How does one create a container



In two steps:

1. from a file called **Dockerfile**, build a docker image, a static snapshot of a filesystem and configuration.
2. run the image to create a live container.

An example Dockerfile:

```
From ubuntu:16.04           # starts from a base 16.04 image
RUN apt-get update && apt-get install -y python3 # create container, run command, commit
RUN mkdir app                # create container, run command, commit
COPY date-time.py /app       # copies code of date-time.py from host to /app in image
WORKDIR /app                 # working dir is /app for future build or container instructions,
                              # baked in the image metadata.
CMD python3 date-time.py     # The command to run in the container, only one allowed
```

To build an image:

```
docker build -t my-python-app .
```



Active Modification (e.g., RUN):

- Spins a temporary container.
- Runs the command inside the container within a default shell
`/bin/sh -c "mkdir /app"`
- Commits the changes to the filesystem to the next layer of the image.

Passive Setup (e.g., COPY, WORKDIR, ENV):

- Does not create a container.
- **COPY** Copies files from the host file system into the image.
- **ENV** Sets environment variables. Also baked into the image as meta data.
- **WORKDIR** Sets working directories for subsequent instructions, records this as meta data in the image.

Container Startup Instruction (CMD):

- Specifies the default command that runs when a container starts
- Not executed during build; saved in image metadata for runtime



Key Dockerfile commands and their usage:

- **ENV:** Set environment variables **inside the container**.
 - Example: ENV CXX=/usr/bin/g++
- **ARG:** Define **build-time** variables.
 - Example: ARG VERSION=16.04
Used as: FROM ubuntu:\$VERSION.
- **VOLUME:** Declare persistent storage location for **/app** in the container.
 - Example: VOLUME /app
- **EXPOSE:** Declare which port the apps running in the container are expected to listen on.
 - Example: EXPOSE 8080
- **USER:** Run the container as a specific non-root user.
 - Example: USER as
- **SHELL:** Change the default shell for RUN instructions.
 - Example: SHELL ["/bin/bash", "-c"]



Containerization:

- When you run (create and start) a container using Docker commands, the Docker Engine
 - allocates resources,
 - sets up the container's isolated environment, and
 - launches the process specified in CMD or in the run command.
- The process runs on the host's kernel.
- Containers start quickly and use fewer resources because they don't need to boot an operating system.



- **docker run** creates a new container from an image and starts executing inside it:

```
docker build -t my-python-app .  
docker run -it --name my-python-container my-python-app bash
```

-i: Keeps the standard input of the container process (bash) (**stdin**) open.

-t: Allocate a pseudo-TTY and attach it to the linux terminal

Together **-it** allows you to interact with the container process, as if it were a normal terminal session.

--name my-python-container: Assigns a custom name to the container.

- **docker run -it --mount type=bind,source=<host_path>,target=<container_path>:**

```
docker run -it --name my-python-container --mount type=bind,source=/home/as/data,target=/app/data  
my-python-app bash
```

Mount a host directory **/home/as/data** inside the container as **/app/data**.

```
docker run -it --name my-python-container --mount type=volume,source=myvol,target=/app/data  
my-python-app bash
```

Create a persistent storage for **/app/data** in the docker volume **myvol**.

Useful docker run options



- **-u <user>:**

```
docker run -u root my-python-app whoami
```

Run the container as a specific user (e.g., root).

- **-e <KEY=VALUE> :** Set environment variables inside container.

```
docker run -e CXX=/opt/gcc-4.7.2/bin/g++-4.7.2 my-python-app
```

Set environment variables inside container.

- **-w <path>:**

```
docker run -w /app my-python-app pwd
```

Set the working directory inside the container before executing CMD.

- **--cap-add <capability>:**

```
docker run --cap-add=SYS_PTRACE my-python-app
```

Grant extra Linux capabilities e.g. **SYS_PTRACE** for examining other processes inside the container.



Inspecting and Managing Containers:

- `docker ps -a`: List all containers (running + exited).
- `docker images`: List locally available Docker images.

Running Commands in Containers:

- `docker exec -it <container> <command>`: Run a new additional process inside an already running container—without touching the original one.
- `docker attach <container>`: Attach your terminal to the main running process inside an already running container.

Container Lifecycle Operations:

- `docker stop <container>`: Gracefully stop a running container. Note: A container also stops when the main process exits.
- `docker start <container>`: Start a previously stopped container. It wakes up and launches (or relaunches) the container's original command (from CMD or ENTRYPOINT).
- `docker rm <container>`: Remove (delete) a stopped container.



What is Docker Compose?

- A tool that helps you run and manage multiple Docker containers at once.
- You describe your setup in a simple text file called **docker-compose.yml**.
- It handles containers, networks, and data storage (volumes) all together as one package.

Key Advantages:

- Makes it easy to start, stop, and rebuild groups of containers with simple commands.
- Lets you run many containers together using just one command, like **docker-compose up**.
- Automatically connects containers through an isolated network, so they can talk to each other by service name.

Docker compose: The setup



`docker-compose.yml`:

```
version: '3.8'
services:
  app:
    build:
      context: .
      dockerfile: MyDockerfile
    depends_on:
      db:
        condition: service_healthy # Wait for
                                   # healthy DB
```

```
db:
  image: mysql:8
  volumes:
    - mysql-data:/var/lib/mysql
  environment:
    MYSQL_ROOT_PASSWORD: example
    MYSQL_DATABASE: mydb
  healthcheck:
    test: [
      "CMD-SHELL",
      "mysqladmin ping -h 127.0.0.1 -u root -p$MYSQL_ROOT_PASSWORD"
    ]
    interval: 5s
    retries: 10
  command:
    - "--bind-address=0.0.0.0"
  volumes:
    mysql-data:
```

- Sets up a private network so **app** can reach **db** by simply using the service name **db**.



MyDockerfile

```
FROM python:3.10-slim
WORKDIR /app
COPY docker-compose-python-script.py /app/
RUN pip install mysql-connector-python
CMD ["bash", "-c", "python /app/docker-compose-python-script.py && tail -f /dev/null"]
```

Docker compose: Python script to connect to MySQL



docker-compose-python-script.py:

```
import mysql.connector
conn = mysql.connector.connect(                                # Connect to MySQL database
    host="db",
    user="root",
    password="example",
    database="mydb",
)
cur = conn.cursor()                                           # Create a cursor

cur.execute("USE mydb;")
cur.execute("DROP TABLE IF EXISTS test;")
cur.execute("CREATE TABLE test (id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(50));")
cur.execute("INSERT INTO test (name) VALUES ('Alice'), ('Bob');")
conn.commit()

cur.execute("SELECT * FROM test;")                             # Query the data
rows = cur.fetchall()
for row in rows:
    print(row)

cur.close()                                                    # Close everything
conn.close()
```



- **docker-compose up**
 - Builds (if necessary) and starts containers.
 - Use **-d** to run in detached mode.
- **docker-compose build --no-cache**
 - Forces a rebuild of images without using any cached layers.
 - Useful after changing the Dockerfile or dependencies.
- **docker-compose down**
 - Stops and removes containers, networks, and optionally volumes.
 - Cleans up all resources created by **up**.
- **docker-compose logs**
 - Shows logs from all services.
 - Use **-f** to follow live logs.



- Docker makes it easy to package, distribute, and run applications in isolated environments called containers that run on a host system.
- Containers offer filesystem, process, network, and security isolation from the host.
- Starting from a Dockerfile, we first build an image, a static snapshot of the filesystem and configuration needed to create a container.
- Images are then run to create containers.
- Docker Compose lets us define and manage multi-container applications with simple YAML files.
- These tools enable a reproducible and scalable software development.



- [dockerdocs](#) – the definitive site for learning and reference.
- [Play with Docker](#) – to experiment in your browser, no installation needed.
- [Getting Started with docker](#) – A learning platform to sharpen your skills.

Recommended Books:

- **Docker Deep Dive: Zero to Docker in a single book by Nigel Poulton. Publishdrive, 2023.**
- **Using Docker: Developing and Deploying Software with Containers by Adrian Mouat. O'Reilly Media, 2016.**